

Artificial Intelligence-[SESIZC444]

CS 7- M4-Indroduction to Learning

By
Prog. Brahmanadu Kakarla

BITS Pilani
Pilani Campus


Start
Recording

Introduction to Learning

- Supervised - Induction by decision trees, Unsupervised Learning, Reinforcement Learning
- Neural Network
- Deep Learning – CNN , RNN
- Build a model using reinforcement learning for given data
- HW: - Identify features for a problem to implement reinforcement learning

Why Learning

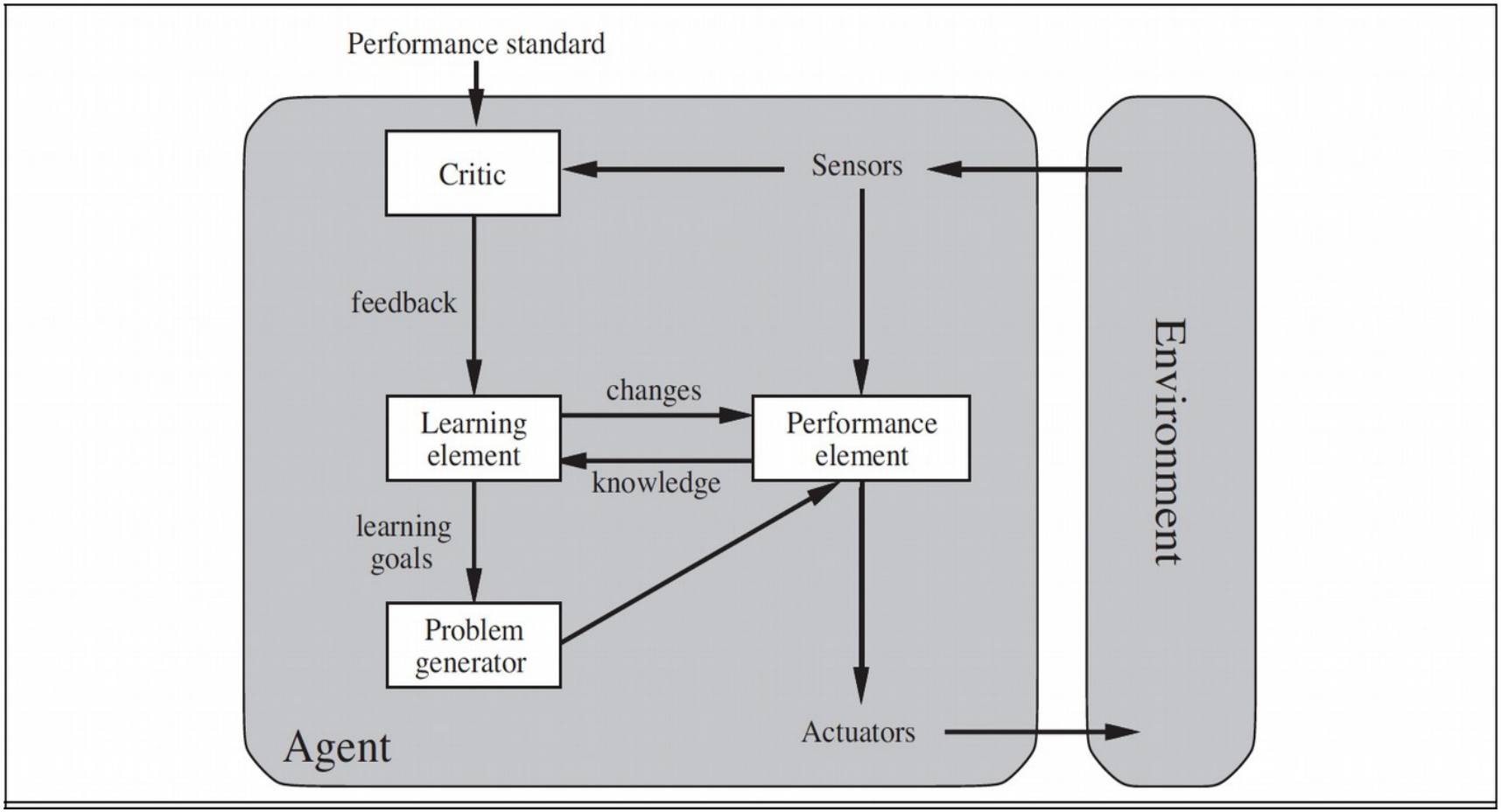
An agent is learning if it improves its performance on future tasks after making observations about the world
Learning happens through observations, i.e., data

1. The agent designer cannot anticipate all possible world states the agent need to handle and code them in the agent environment
    1. E.g., A robot designed to navigate mazes need to learn the layout of new maze whenever encountered

2. The agent designer cannot anticipate all changes over time
    1. E.g., Stock market prediction

3. Sometimes the agent designer have no idea how to program a solution
    1. E.g., Stock market prediction.

Learning Agent



Learning Agent Components

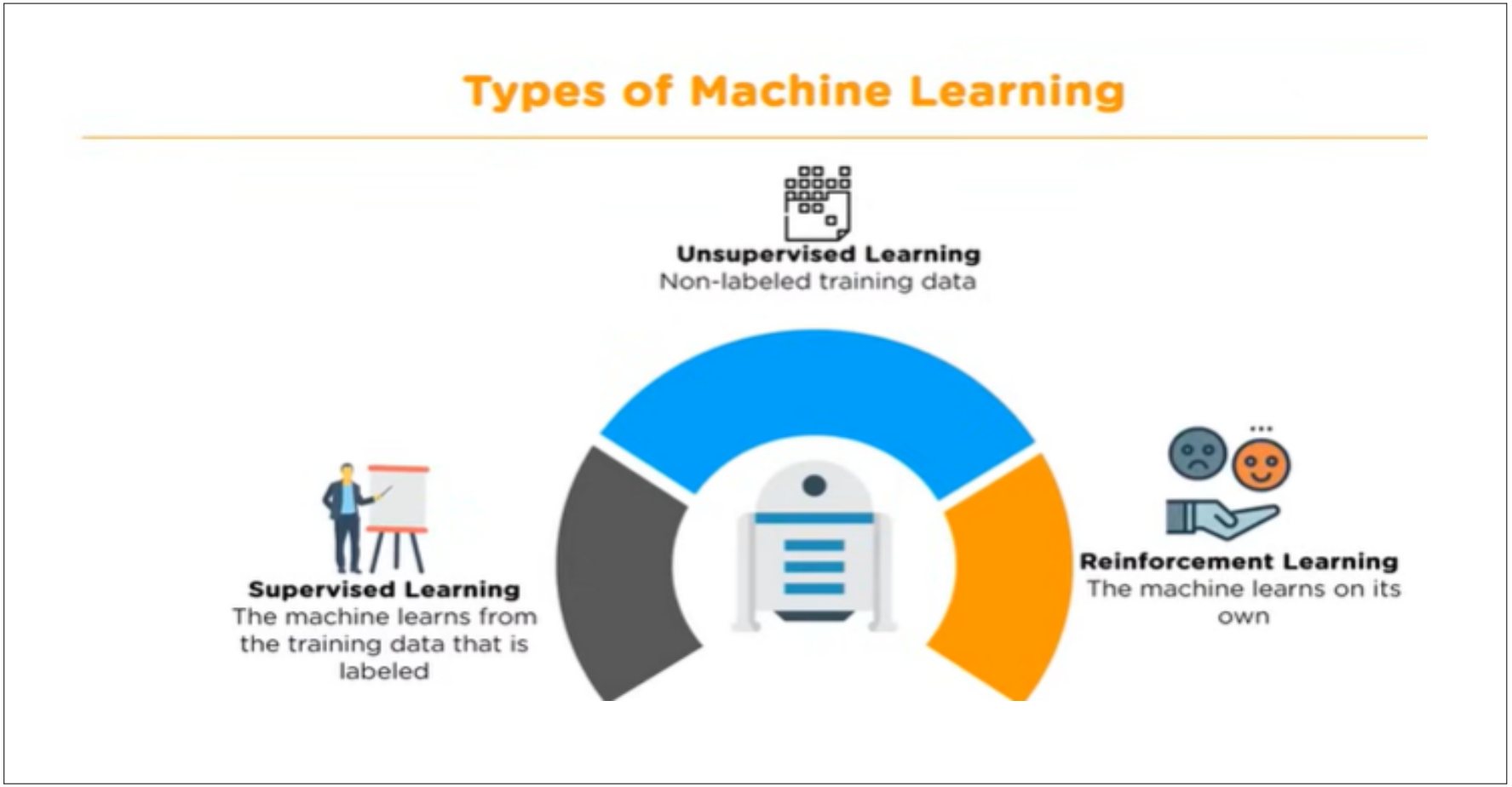
Performance Element – taking a decision of action based on percepts

Learning Element – Make the performance element select better actions such that the utility function is optimized

Critic – Provides feedback on the actions taken

Problem Generator – Make the Performance Element select sub-optimal actions such that you would learn from unseen actions.

Learning


Types of Machine Learning
Unsupervised Learning
Non-labeled training data
Supervised Learning
The machine learns from the training data that is labeled
Reinforcement Learning
The machine learns on its own

Supervised Learning / Predictive models:

- **Supervised Learning** is a type of learning where:
- We **learn from past data**
- **Correct answers (labels)** are already given
- The goal is to **predict future outcomes.**
- The model is told **what to learn** and **how to learn** using labeled examples

Supervised Learning / Predictive models:

Customer Churn Prediction

Problem Statement:

A marketing company wants to identify which customers are **likely to stop using their service** in the future.

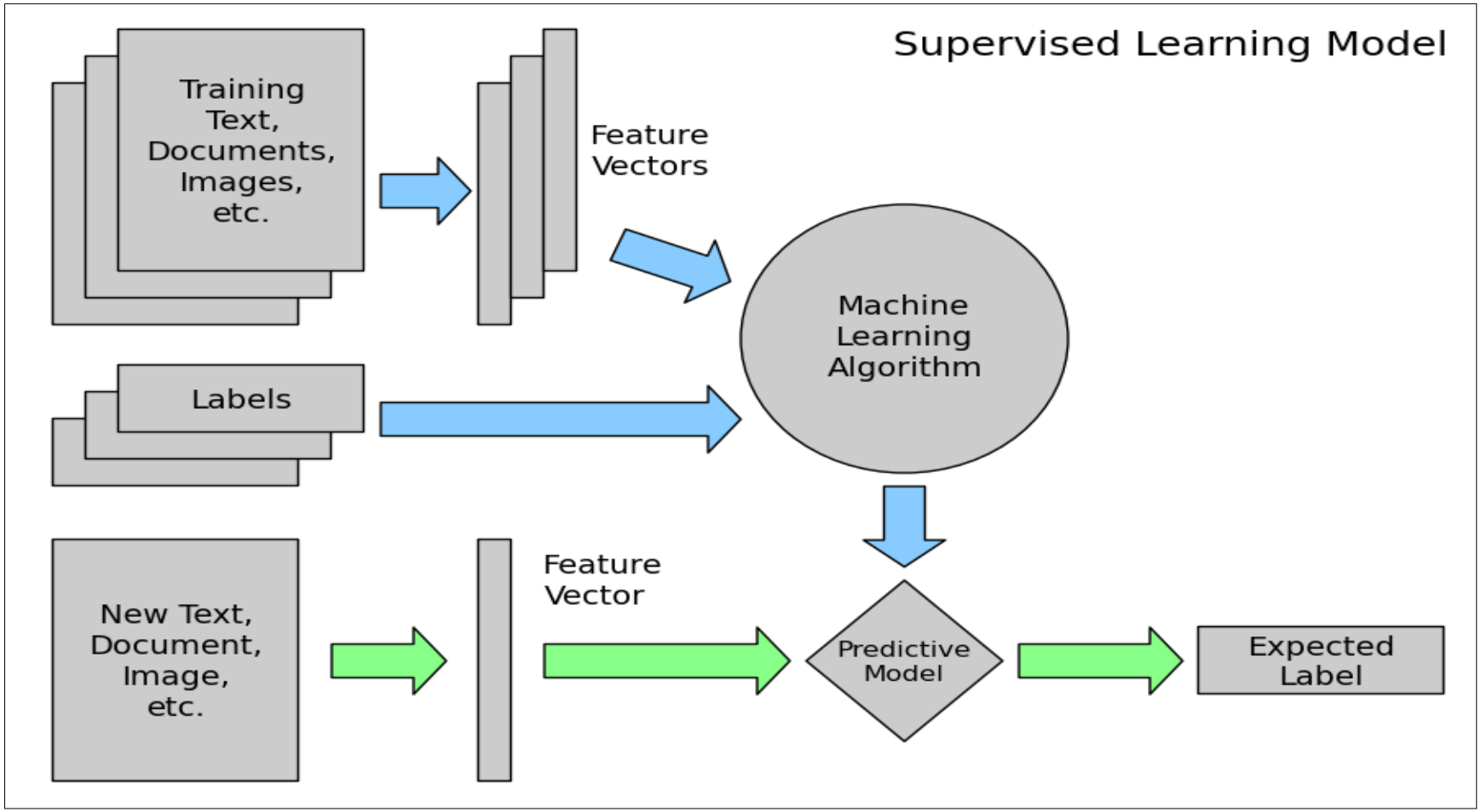
Given Data (Labeled):

Customer	Usage	Complaints	Churn
C1	High	Low	No
C2	Low	High	Yes
C3	Medium	Low	No

Task:

Predict whether a **new customer** will churn or not.

- Output label (**Churn: Yes/No**) is already known in training data
- Model learns from past examples to predict future customers


Supervised Learning Model
Training Text, Documents, Images, etc.
Feature Vectors
Machine Learning Algorithm
Labels
New Text, Document, Image, etc.
Feature Vector
Predictive Model
Expected Label

Unsupervised learning / Descriptive models

- Learns **patterns from data without labels**
- **No target output** is given
- Groups similar data items together

Simple Real-World Examples

Retail Example

- Data: Customer purchase history
- Goal: Find **which products are often bought together**
- Output: Customer/product **groups (clusters)**

Healthcare Example

- Data: Patient symptoms and reports
- Goal: Find **diseases that commonly occur together** (e.g., with diabetes)
- Output: Disease **clusters**

Key Algorithm:K-Means Clustering

Reinforcement Learning(RL)

innovate achieve lead

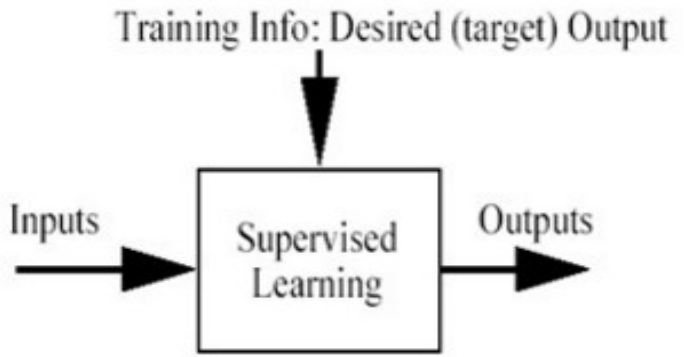
- A learning method where an **agent learns by interacting with an environment**
- The goal is to **maximize performance (reward)**
- The agent **learns continuously from experience**, not from labeled data
- **Less human intervention** is required over time.
- Learn **what action to take** in a situation based on **reward or penalty**

Example

Robot learning to navigate a path

Game AI learning to win by trial and error

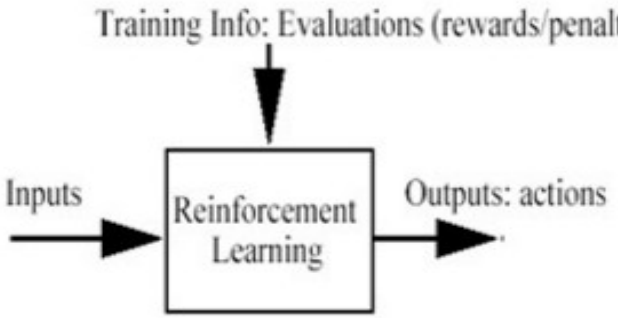
Supervised Learning



Error = (target output - actual output)

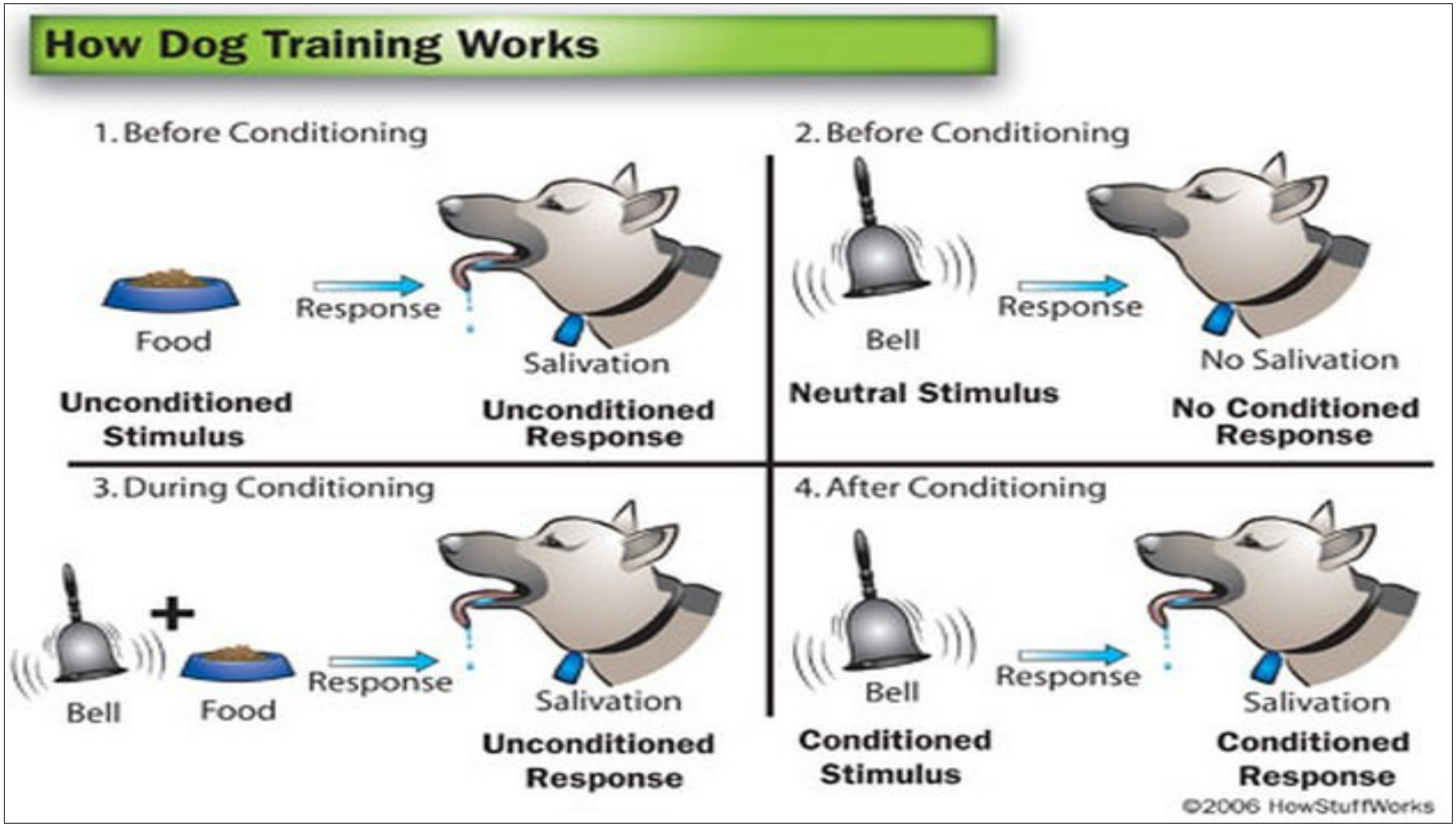
Input is an istance, output is a classification of the istance

Reinforcement Learning



Objective: Get as much reward as possible

Input is some "goal" output
Is a sequence of actions to meet the goal


How Dog Training Works
1. Before Conditioning
Food
Response
Salivation
Unconditioned Stimulus
Unconditioned Response
2. Before Conditioning
Bell
Response
No Salivation
Neutral Stimulus
No Conditioned Response
3. During Conditioning
Bell
Food
Response
Salivation
Unconditioned Response
4. After Conditioning
Bell
Response
Salivation
Conditioned Stimulus
Conditioned Response
©2006 HowStuffWorks

Supervised learning	Unsupervised learning	Reinforcement learning
Input/Output pairs	Input only	Input & Critic
Learning phase v.s. acting phase	Learning phase v.s. acting phase	Learning and acting simultaneously – online learning
		Explicitly explore the world and learn by trial and error

Learning Element - Feedback

There are three (3) types of feedback that determine the learning form

1. **Supervised – Feedback** will be provided for every action suggesting the correct action
    1. E.g., In a Spam Classifier, if the agent tags an email as Spam, the Feedback says whether it is actual Spam or Not. Can be multiple classes.
2. **Reinforcement – Feedback** will be provided for a collection of actions suggesting whether the end state is desirable (reward) or not (punishment)
    1. Doesn't mention the corrective actions
    2. E.g., Two points at the end of a chess game would tell the agent it did something right but the agent has to figure out which actions are responsible.
3. **Unsupervised – No feedback.** The agent need to figure out patterns.
    1. Clustering related news articles in Google news

Supervised Learning

Given a training set of N example input-output pairs

$$(x_1, y_1), (x_2, y_2), (x_3, y_3) \ldots \ldots \ldots (x_n, y_n)$$

where each y_j was generated by an unknown function f,

Such that $y_j = f(x_j)$

Task: Identify a function h that can approximate the true unknown function f

Function h is called a hypothesis . Learning is the task of searching through the space of possible hypothesis using the given examples (train set) such that. The hypothesis performs well on a unseen set of observations, called test set

Generalization: The ability of a hypothesis to perform the correct prediction on unseen observations. The higher the better

Decision Tree

Decision Tree is a tree shaped diagram used to determine a course of action. Each branch of the tree represents a possible decision, occurrence or reaction

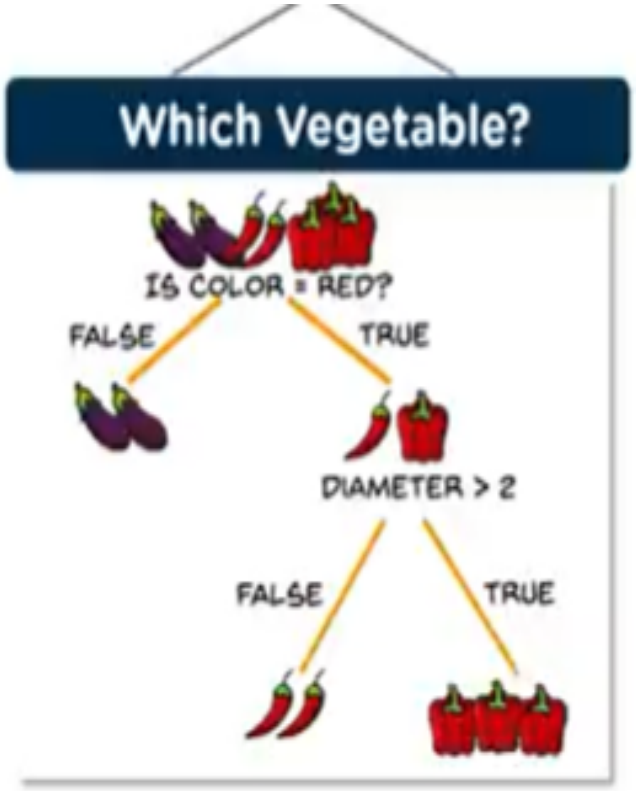
It is the most simple yet most powerful algorithm in machine learning
Let us first describe the Hypothesis Space and then learn a good hypothesis

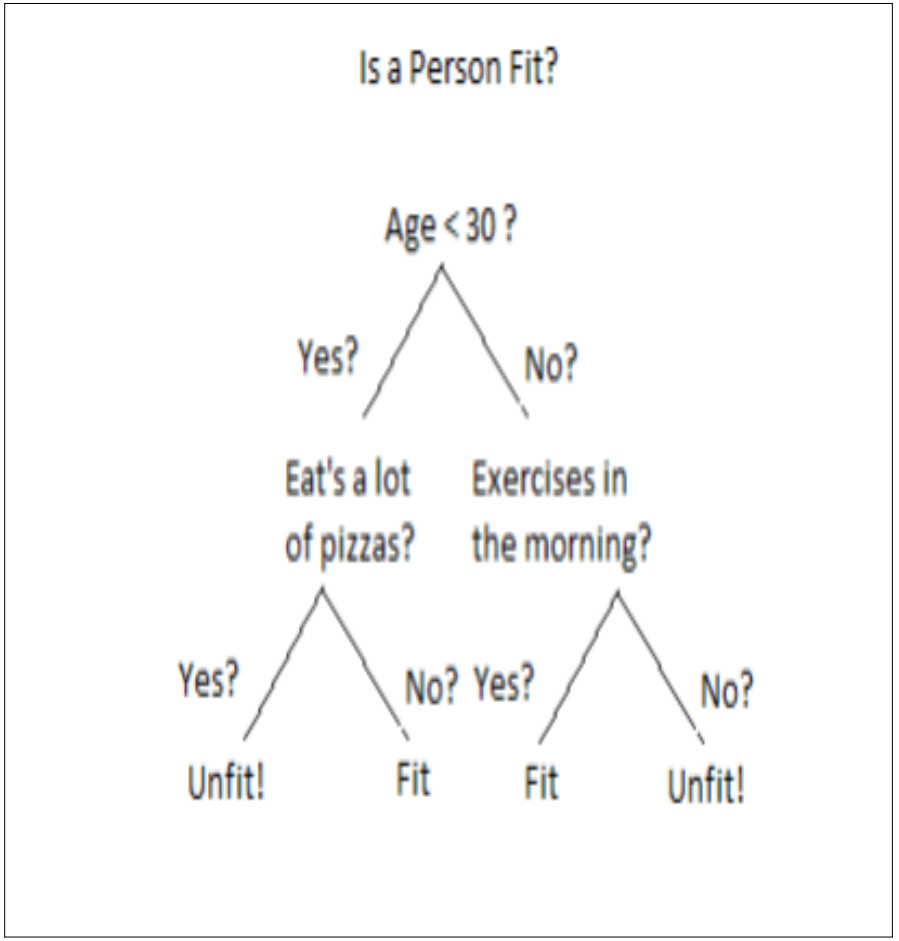
Input: A vector of attribute values **Output:**
A ”decision” - single output value E.g., [1.2,
3, ’a’, 4] -> “spam”

Decision Tree: A tree consists of nodes and directed edges where decisions are made at each node on selecting a child branch. Decisions are made by a test on one of the input attribute values. Each leaf node will be the return value.

Decision Tree

DECISION TREE





DECISION TREE EXAMPLE

- Which feature will be at the root node of the decision tree trained for the following data? In other words which attribute makes a person most attractive?

Height	Hair	Eyes	Attractive?
small	blonde	brown	No
tall	dark	brown	No
tall	blonde	blue	Yes
tall	dark	Blue	No
small	dark	Blue	No
tall	red	Blue	Yes
tall	blonde	brown	No
small	blonde	blue	Yes

STEP 1

Height	Hair	Eyes	Attractive
small	blonde	brown	No
tall	dark	brown	No
tall	blonde	blue	Yes
tall	dark	blue	No
small	dark	blue	No
tall	red	blue	Yes
tall	blonde	brown	No
small	blonde	blue	Yes

Step 2: Count Yes / No

- Yes = 3
- No = 5
- Total = 8

ENTROPY

Entropy

Definition

Entropy is a measure of **uncertainty or impurity** in a dataset.

- High entropy → data is mixed (more disorder)
- Low entropy → data is pure (mostly one class)

Formula

$$Entropy(S) = -\sum p_i \log_2 p_i$$

For binary classification:

$$Entropy(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

Simple Example

If a dataset has:

- 50% Yes, 50% No

$$Entropy = 1 \quad \text{(maximum uncertainty)}$$

If all examples are **Yes**:

$$Entropy = 0 \quad \text{(no uncertainty)}$$

INFORMATION GAIN

Information Gain

Definition

Information Gain measures how much **entropy is reduced** after splitting a dataset on an attribute.

- Higher Information Gain → better attribute for splitting
- Used to choose the **root node** in a decision tree

Formula

$$IG(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

where:

- S = original dataset
- S_v = subsets after split on attribute A

Simple Example

If:

- Entropy before split = 1
- Entropy after split = 0.4

$$IG = 1 - 0.4 = 0.6$$

Generalization

Goal: Build a model which is accurate enough on train set and is generalizable (performs similarly) to unseen examples

Can't keep generating nodes as it performs poorly on test sets

- The goal is to build a model that **performs well on both training data and unseen data**
- A good model should **not just memorize** the training set
- Creating too many nodes (over-complex model) leads to **poor performance on test data**

Generalization

Overfitting

Overfitting occurs when a model learns the training data too well and fails to generalize to new data.

Overfitting (Simplified)

- The model performs **very well on training data**
- But performs **poorly on unseen (test) data**
- Happens when the model becomes **too complex**

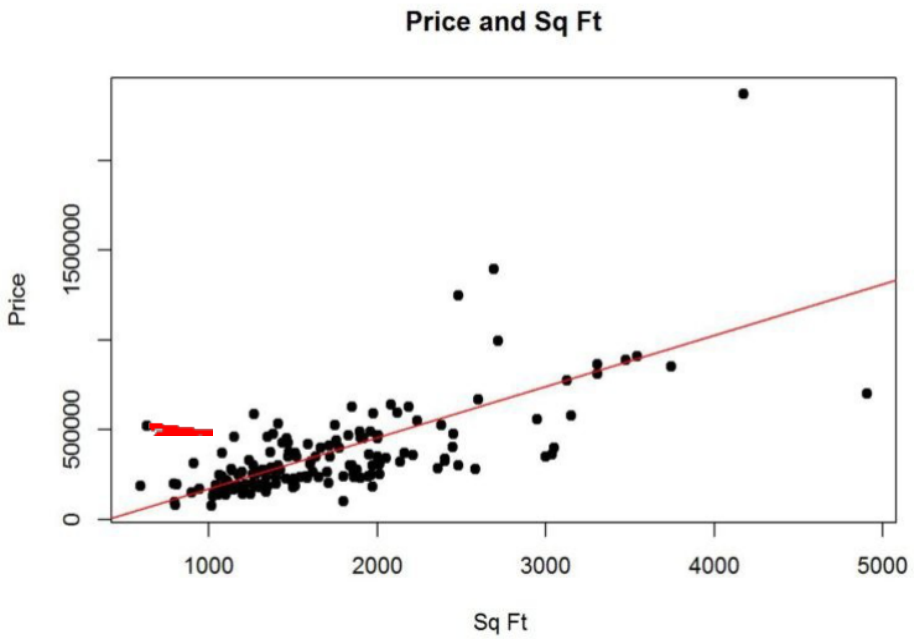
How to Reduce Overfitting (Decision Trees)

Pruning

- Remove **irrelevant or noisy nodes**
- Build the full decision tree
- Identify nodes whose children are **only leaf nodes**
- If a node does **not improve accuracy**, remove it

Regression

Regression is a **supervised learning technique** used to predict a **continuous numerical value**.E.g., Predict the prices of houses based on the sqft of house



Types of Regression

Type	Description
Linear Regression	One input → one output
Multiple Linear Regression	Multiple inputs → one output
Polynomial Regression	Curved relationship
Logistic Regression	Classification (output 0 or 1)

Linear Regression

Univariate Linear Regression

A linear function with input x and output h represented as

$$y = h_w(x) = w_1 x + w_0$$

Learning the weight vector $[w_1, w_0]$ from the training examples that best fits the data is referred to as linear regression

E.g., Predicting house prices from sqft alone is univariate linear regression

Multivariate Linear Regression

A natural extension of univariate linear regression is by having a weighted linear combination of multiple variables

$$y_j = h_w(x_j) = w_0 + \sum_i w_i x_{j,i}$$

for each example x_j

E.g., Predicting house prices as a function of [sqft, location, number of bed rooms]

Simple Linear Regression

lead

Model Equation

$$y = mx + c$$

Where:

- x = independent variable (input)
- y = dependent variable (output)
- m = slope (rate of change)
- c = intercept

Example 1: Simple Linear Regression (Step-by-Step)

Given Data

Study Hours (x)	Marks (y)
2	40
4	80

Step 1: Calculate Slope (m)

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{80 - 40}{4 - 2} = 20$$

Step 2: Calculate Intercept (c)

$$40 = 20(2) + c \Rightarrow c = 0$$

Step 3: Regression Equation

$$\boxed{y = 20x}$$

Step 4: Prediction

If a student studies 3 hours:

$$y = 20 \times 3 = 60$$

✔ Predicted marks = 60

6. Interpretation of Regression Line

- **Slope (m)** → How fast output changes with input
- **Intercept (c)** → Output when input is zero
- Regression line gives **best fit** to data

---

7. Error in Regression

Error (Residual)

$$Error = Actual\ value - Predicted\ value$$

Regression tries to **minimize total error.**

---

8. Cost Function (Mean Squared Error)

$$MSE = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

Where:

- y_i = actual value
- $\hat{y}_i$ = predicted value

✔ Smaller MSE → better model

Multiple Linear Regression

lead

Equation

$$y = b_0 + b_1 x_1 + b_2 x_2 + \dots$$

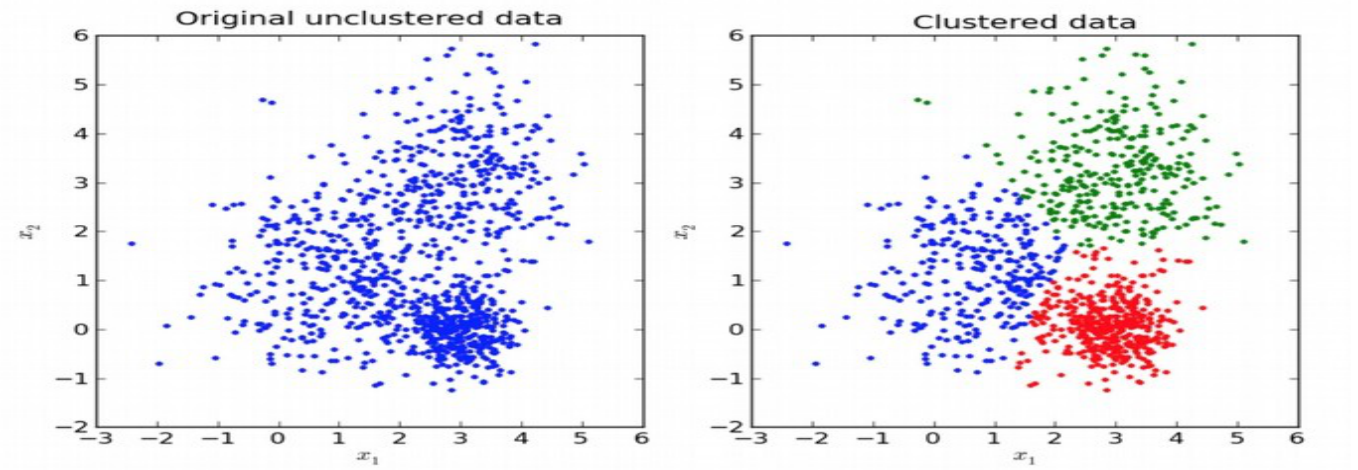
Example Inputs

- House size
- Number of bedrooms
- Location

Unsupervised Learning : Clustering

To cluster examples in data that are similar to each other. Similar entities share similar attributes which means partial redundancy. By clustering similar entities and representing them as a single entity would help to disentangle the variation in data

E.g., finding groups of customers with similar behavior given the customer data with their properties and past buying records



K-means Clustering

K-means algorithm divides the training set into k different clusters $[\mu^{(1)}, \mu^{(2)}, \ldots\ldots \mu^{(k)}]$ that are near to each other. K-means algorithm works by randomly initializing k different centroids to different values, then altering between 2 steps until convergence

In one step, each training example is assigned to cluster *i*, where *i* is the index of nearest centroid $\mu^{(i)}$

In another step, each centroid $\mu^{(i)}$ is updated to the mean of all training examples $x^{(j)}$ assigned to cluster *i*

Reinforcement Learning



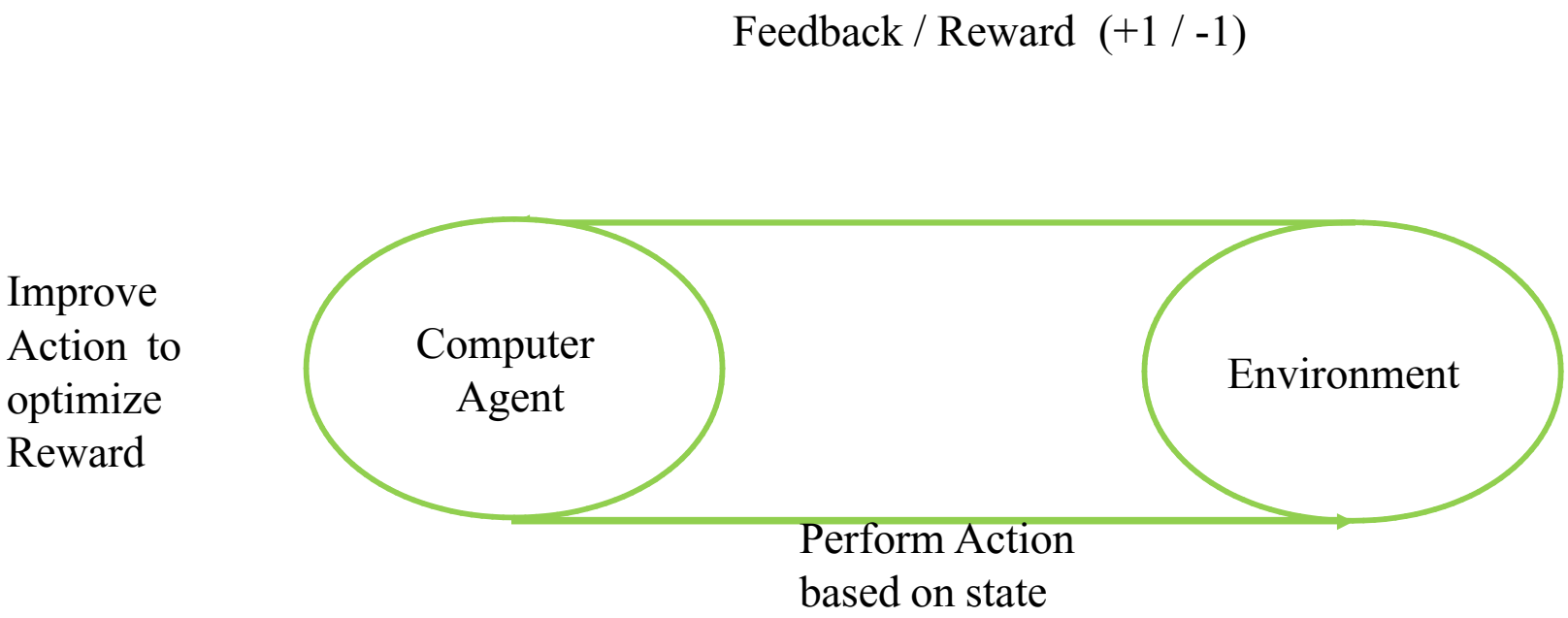
The frequency of rewards varies with the problem



A Policy is a function that maps inputs to actions
The goal of Reinforcement Learning – Identify an optimal policy that maximizes the expected total reward
Task: Use observed rewards to learn an optimal policy for the environment

Reinforcement Learning in nutshell: Imagine playing a new game and after 100 moves your opponent says you lost the game.

ML: Reinforcement Learning



Agent and Components of Agents

Agents :

The agent makes the decision of what action to make

Agent is allowed to use any observation from environment and internal rules it has

At each step 't' the agent:

Executes an action A_t
Receives observation O_t
Receives scalar Reward R_t

Components of Agent

- Policy
- Value Function
- Model

Policy

Policy defines the behavior of the agent

If you are lazy, you would look for shortcuts to reach the destination soon or you may not even reach the destination if your calculations are wrong or if there are obstacles in the way

Stochastic Policy:-

For every decision you have a probability given the current state. Over experience you want to optimize this probability distribution so that you get a higher chance of success while finding shortcuts

If you were taking the methodical approach, you would be knowing what direction to chose at any point of time

Deterministic Policy:- a function that maps states to actions

here, '$\prod$' is just the policy function that simply gives an output state given your current state

Value Function

- Value Function defines how good it is to be in a particular state.

- Calculating which path will be more feasible for you by weighing both the reward and obstacles is defined by Value Function

$$V_\pi(s) = E_\pi[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots | S_t = s]$$

here 's' is the current state

'R' is the reward

'π' is the policy

'γ' is the discount

Model

Model is the agent's representation of the environment

Transition Model:
Model try to predict the dynamics of the environment with the data it has seen till that point

Reward Model:
Based on previous experiences, predicting the reward provided by environment.

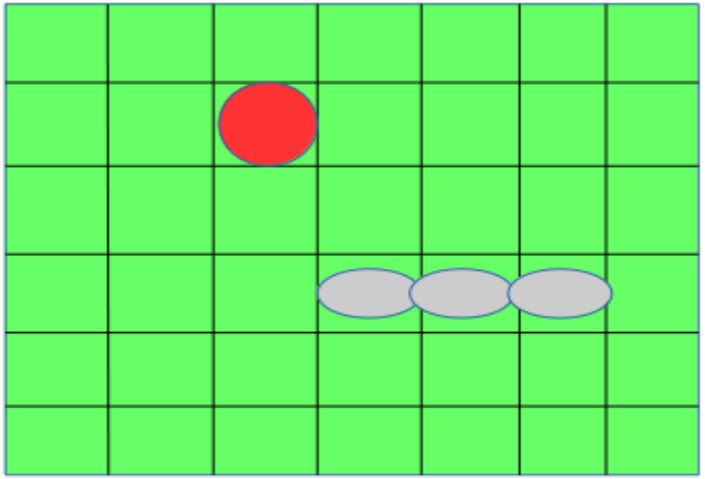
Snake Example

Instructions :

Eat apple(red circle) to get points.

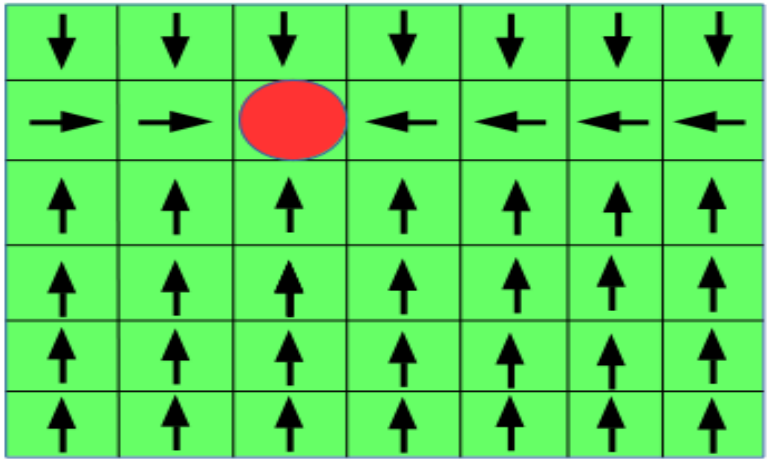
Eat cherries(pink circle to get bonus points) If you hit wall or eat yourself you die.

Controls are UP, DOWN, LEFT, RIGHT



Applying Policy

Snake need to have some behavior to eat the apple



Applying Value Function

In value function we assign every grid a value



Applying Model

Model is just a prediction of the environment variables according to the agent

Initially the snake doesn't have an idea about the surrounding

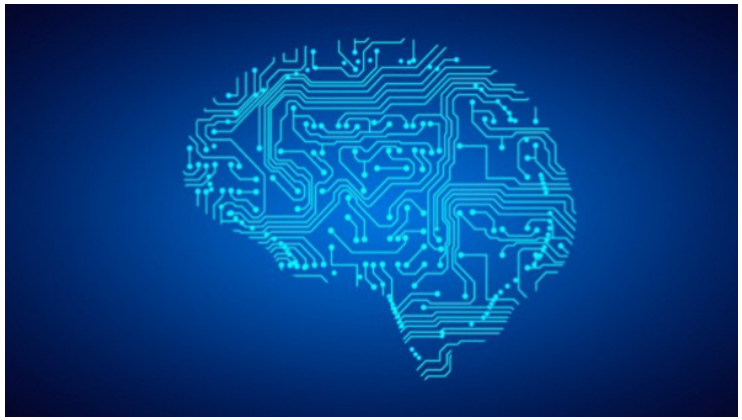
Looking at the policy, it has an idea of which direction to go

Looking at the value function it has an idea as how to maximize the reward for each step

Then the agent takes the decisions.

Artificial Neural Networks (ANN)

How does brain works?
What are the mathematical models of brain activity?



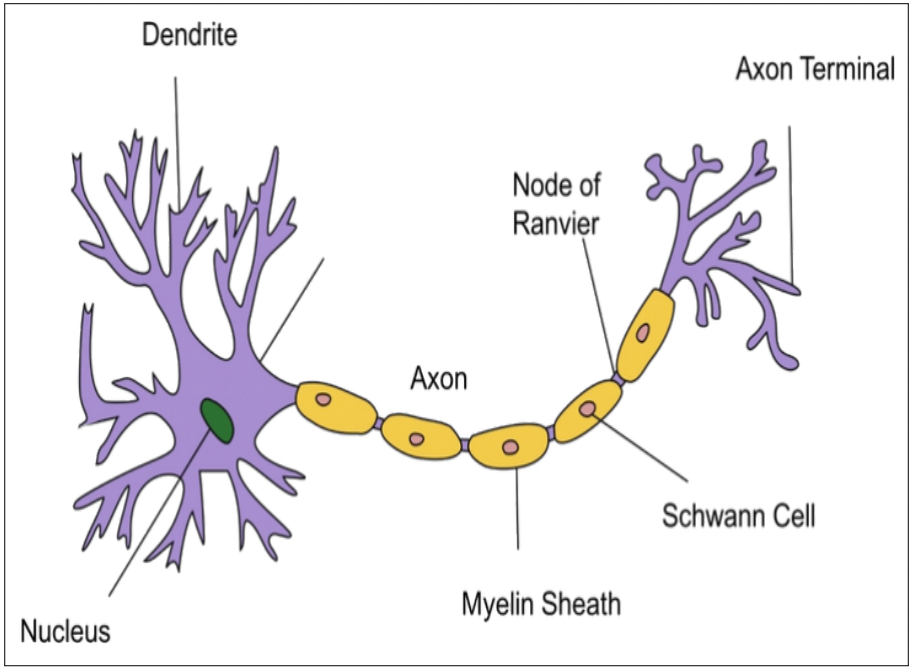
Neurons

The hypothesis that mental activity consists primarily of electrochemical activity in networks of brain cells fires a downward signal when the input signal exceeds a threshold

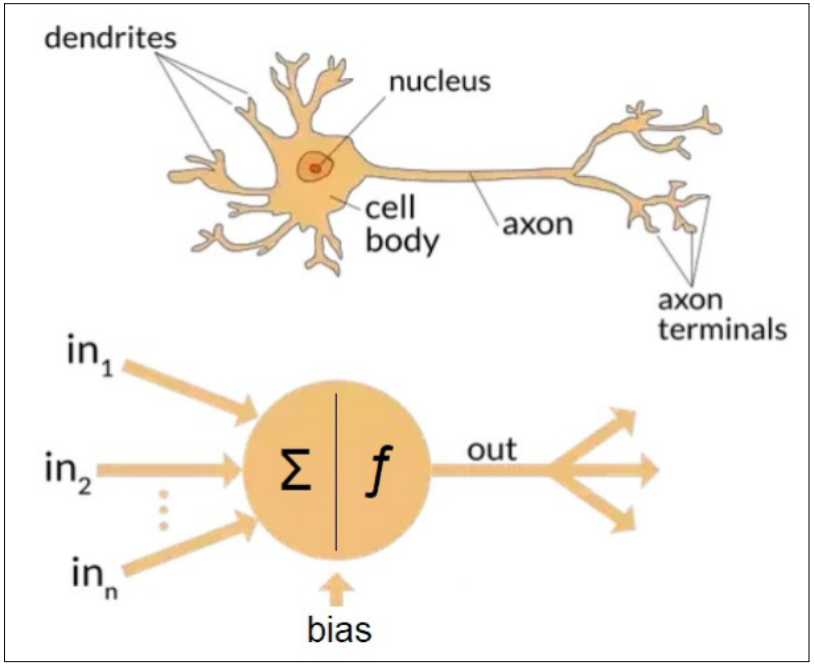
Neural Network

Collection of units connected together, the properties of network are determined by its topology and the properties of neurons .

Neuron



Biological Neuron



Artif ɗ ial Neuron

ANN building blocks and Example ANN

Neuron Units

Basic structures of Neural Networks

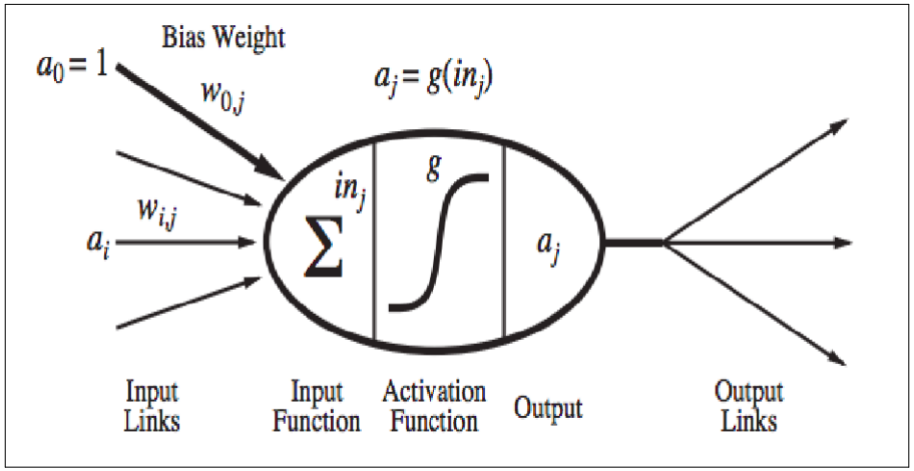
Also known as artificial neurons

Receives signals, processes them and transmit

Links

Units are connected through links

Each link can transmit a signal from one artificial neuron to another



Activation

Transforms the input signal into a restricted output

EXAMPLE ANN

Weight

Each link also has a numeric weight w_{ij} associated with it

Determines the strength and sign of connection

Each unit *j* first computes a weighted sum of its inputs:

Activation Function

An activation function *g* to this sum to derive the output

ANN Activations

Threshold Activation

Linear single layer classifier

The activation function g is typically a hard threshold

Called a Perceptron

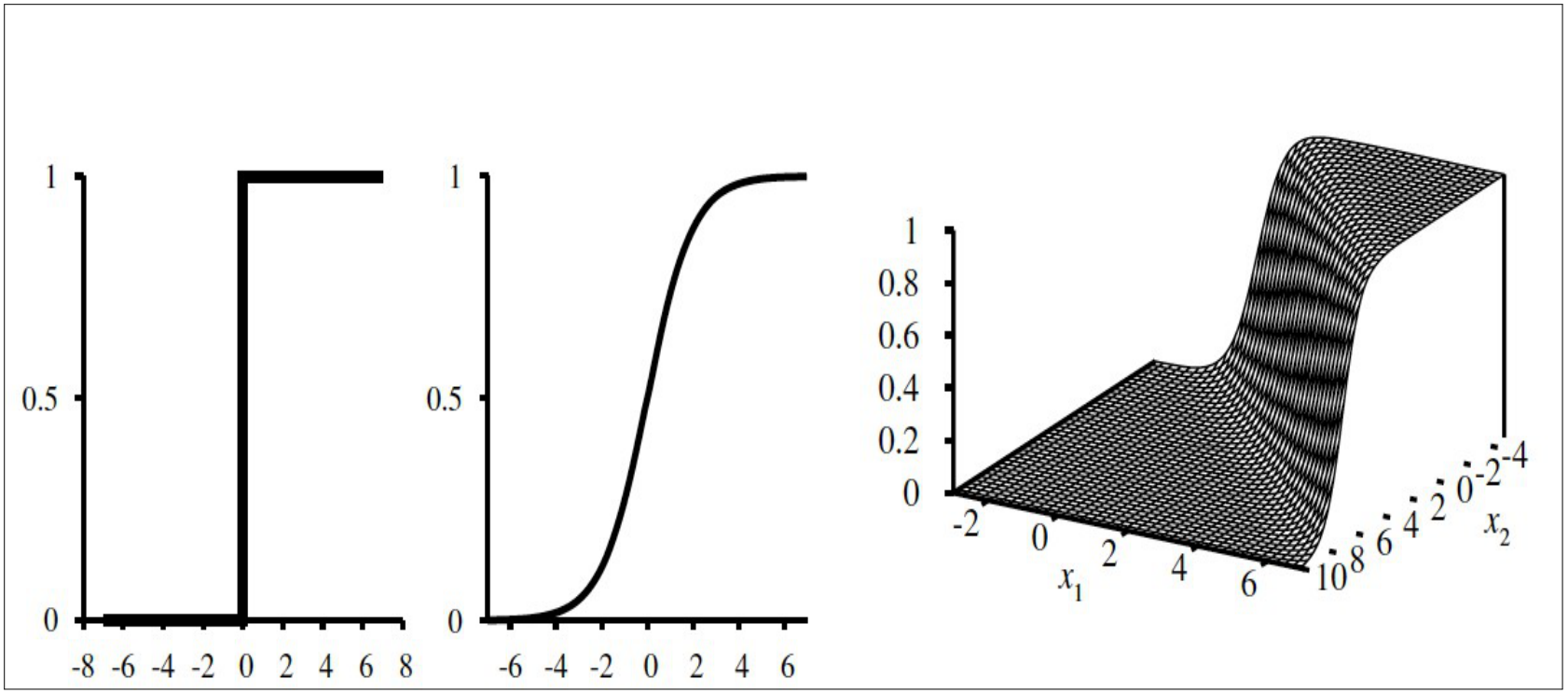
Logistic Function

Also referred to as Sigmoid perceptron

Both of these nonlinear activation function ensure the important property that the entire network of units can represent any nonlinear function

The logistic activation function has the added advantage of being differentiable

ANN Activations



ANN building blocks

Network of neurons:
Connecting the neurons together to form a network

Feed-Forward Network

A network that has connections only in one direction—that is, it forms a directed acyclic graph.

Every node receives input from "upstream" nodes and delivers output to "downstream" nodes; there are no loops

A feed-forward network represents a function of its current input; thus, it has no internal state other than the weights themselves

Layers: Feed-forward networks are usually arranged in layers, such that each unit receives input only from units in the immediately preceding layer

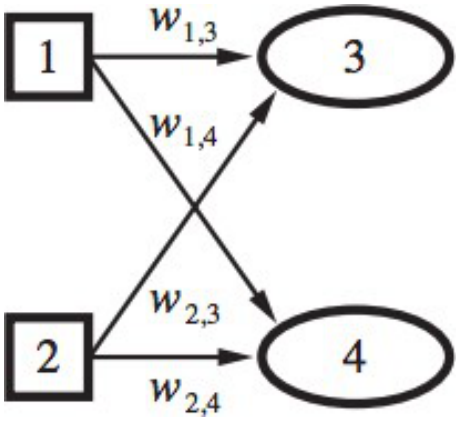
Hidden Units: One or more layers of units that are not connected to the outputs of the network.

Single Layer Feed-Forward Neural Network

A network with all the inputs connected directly to the outputs For Example
Consider addition of two binary digits

Two output units, one for sum and one for carry is needed

The perceptron network is as below



Truth table of the network

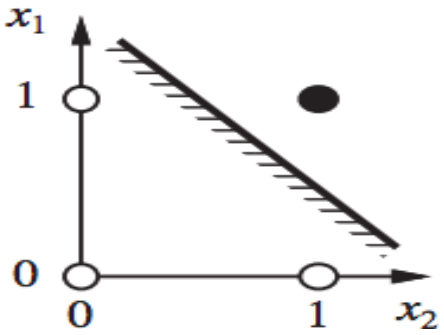
x_1	x_2	y_3 (carry)	y_4 (sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Perceptron Network

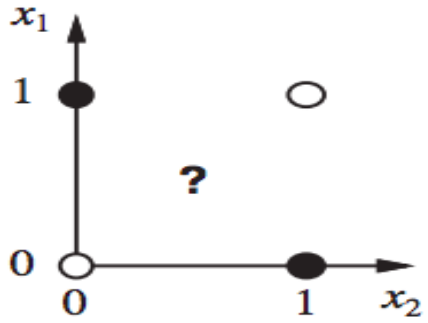
A perceptron network with m outputs is really m separate networks, because each weight affects only one of the outputs

So, there will be m separate training processes

Unit 3 Classifier



Unit 4 Classifier



Unit 3 classifier is logical AND and Unit 4 classifier is logical XOR

Logical AND learns easily with Neural Networks as it is a linear classifiers and can represent linear decision boundaries in the input space

Logical XOR function is difficult to learn as it is not linearly separable

Example for complex function which can be represented with Neural Network easily

Majority Function:- which outputs a 1 only if more than half of its inputs are 1 can be represented by a perceptron with each and

Whereas a decision tree would need exponentially many nodes to represent this function

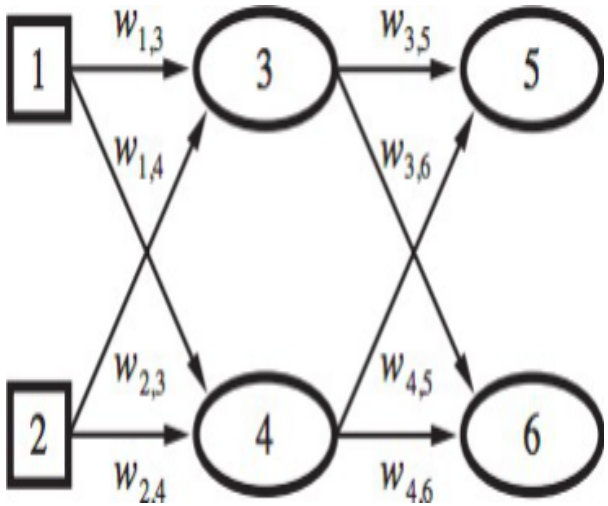
Multi-Layer Feed-forward Neural Network

Single layer Feed-Forward can only represent basic boolean functions

To deal with many other functions, more hidden layers are needed

Example of a Multi-layer Feed-forward network

Given an input vector , the activations of the input units are set to



$$
\begin{aligned}
a_5 &= g(w_{0,5,+} w_{3,5}\, a_3 + w_{4,5}\, a_4) \\
&= g(w_{0,5,+} w_{3,5}\, g(w_{0,3} + w_{1,3}\, a_1 + w_{2,3}\, a_2) + w_{4,5}\, g(w_0 4 + w_{1,4}\, a_1 + w_{2,4}\, a_2)) \\
&= g(w_{0,5,+} w_{3,5}\, g(w_{0,3} + w_{1,3}\, x_1 + w_{2,3}\, x_2) + w_{4,5}\, g(w_0 4 + w_{1,4}\, x_1 + w_{2,4}\, x_2))
\end{aligned}
$$

What is Deep Learning?

A machine learning subfield of learning **representations** of data, where we use algorithms that attempt to learn (multiple levels of) representation by using a **hierarchy of multiple neural layers**

If you provide the system **tons of information**, it begins to understand it and respond in useful ways.

DL is exceptionally effective at **learning patterns**.

Multi-Layer Feed-forward Neural Network

The output expressed as a function of the inputs and the weights

Calculate the derivatives of such expressions w.r.t. the weights The gradient-descent loss-minimization method is used to train the network

Because the function represented by a network can be highly nonlinear—composed, as it is, of nested nonlinear soft threshold functions— neural networks is used as a tool for nonlinear regression.

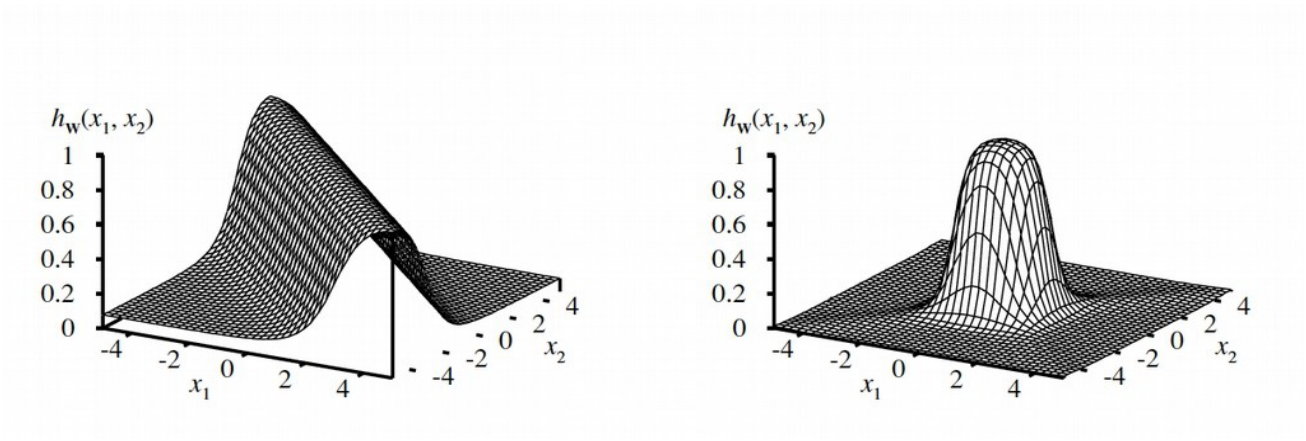
Multi-Layer Feed-forward Neural Network

How do networks generate complicated functions?
Each unit in a sigmoid network represents a soft threshold in its input space

With one hidden layer and one output layer, each output unit computes a soft-thresholded linear combination of several such functions

By adding two opposite-facing soft threshold functions and thresholding the result, a ridge function is obtained

Adding two ridge functions at right angle to each other we can obtain a bump



Learning in Neural Networks

Let be the output vector function and returns a vector for the above multilayer network Let target output will be a vector
The loss function is additive across the components of the error vector The loss is given by

Where the index ranges over nodes in the output layer.
Each term in the final summation is just the gradient of the loss for the output, independent of other outputs

$$\frac{\partial}{\partial w} Loss(\mathbf{w}) = \frac{\partial}{\partial w} |\mathbf{y} - \mathbf{h_w}(\mathbf{x})|^2 = \frac{\partial}{\partial w} \sum_k (y_k - a_k)^2 = \sum_k \frac{\partial}{\partial w} (y_k - a_k)^2$$

Back-Propagation with Example

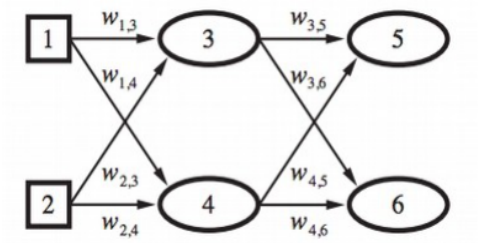
The error at the output layer is $y - h_w$

But to determine the error at the hidden layers, the error from output layer is **back-propagated** to the error in hidden layers

The back-propagation process is as follows:

Compute the Δ values for the output units, using the observed error.
Starting with output layer, repeat the following for each layer in the network, until the earliest hidden layer is reached:
Propagate the Δ values back to the previous layer.
Update the weights between the two layers.

- Where Δ is the error between predicted and target output



Initialize all the weights of the network to some random numbers
Now for each example data point,

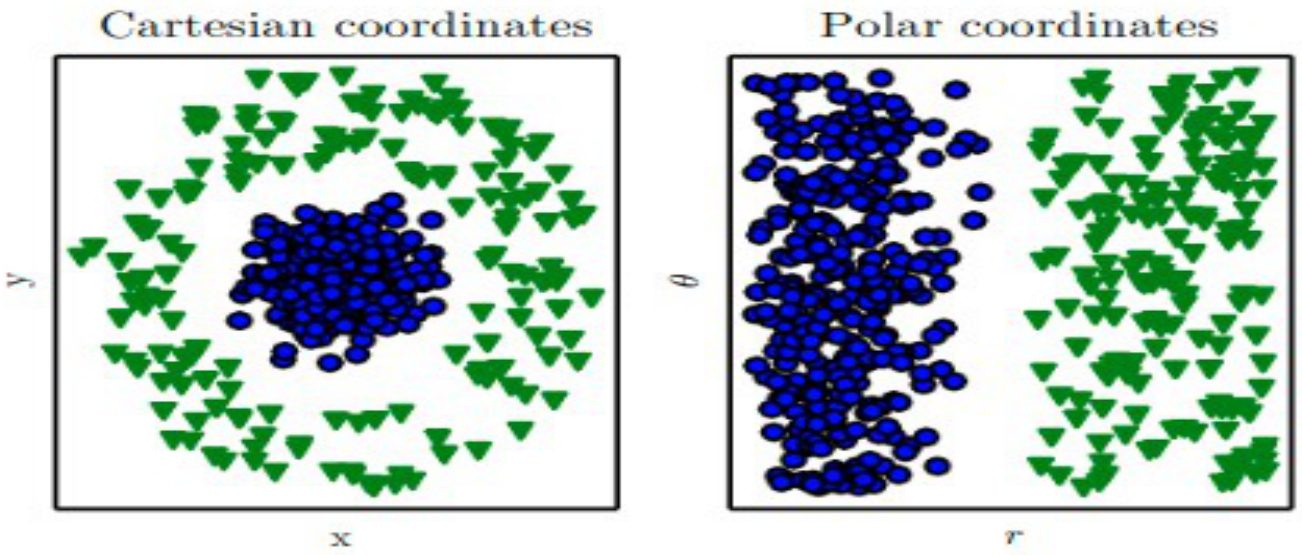
1. Propagate the inputs forward to compute the outputs :
   Initialize the input activations to the input numbers i.e., $\forall\ a_i = x_i$
   In our example, $a_1 = x_1;\ a_2 = x_2$
   Calculate the hidden and output activations using the formula

$$in_j = \sum_{i=0}^{n} w_{i,j} a_i \ .$$

$$a_j = g(in_j) = g\left(\sum_{i=0}^{n} w_{i,j} a_i\right)$$

Background - Machine Learning Example

Suppose we want to separate two categories of data by drawing a line between them in a scatterplot.
In the plot on the left, we represent some data using Cartesian coordinates, and the task is impossible.
In the plot on the right, we represent the data with polar coordinates and the task becomes simple to solve with a vertical line.



Solution

Use Machine Learning to discover not only the mapping from representation to output but the representation as well
This is called Representation Learning
Enable AI systems to rapidly adapt to new tasks with minimal human intervention

Manually designing features for complex task requires a great deal of human effort
The quintessential example of representation learning is autoencoder.

Drawbacks of traditional learning

A major source of difficulty in many real-world AI applications is that factors of variation influence every single example of data we observe

Most applications want us to disentangle the factors of variation and discard the ones we do not care about

Many of these factors of variation, such as speaker's accent (in Speech Recognition), can be identified only using sophisticated, nearly human- level understanding of data

When it is nearly as difficult to obtain a representation as to solve the problem, representation learning does not, at first glance, seem to help us

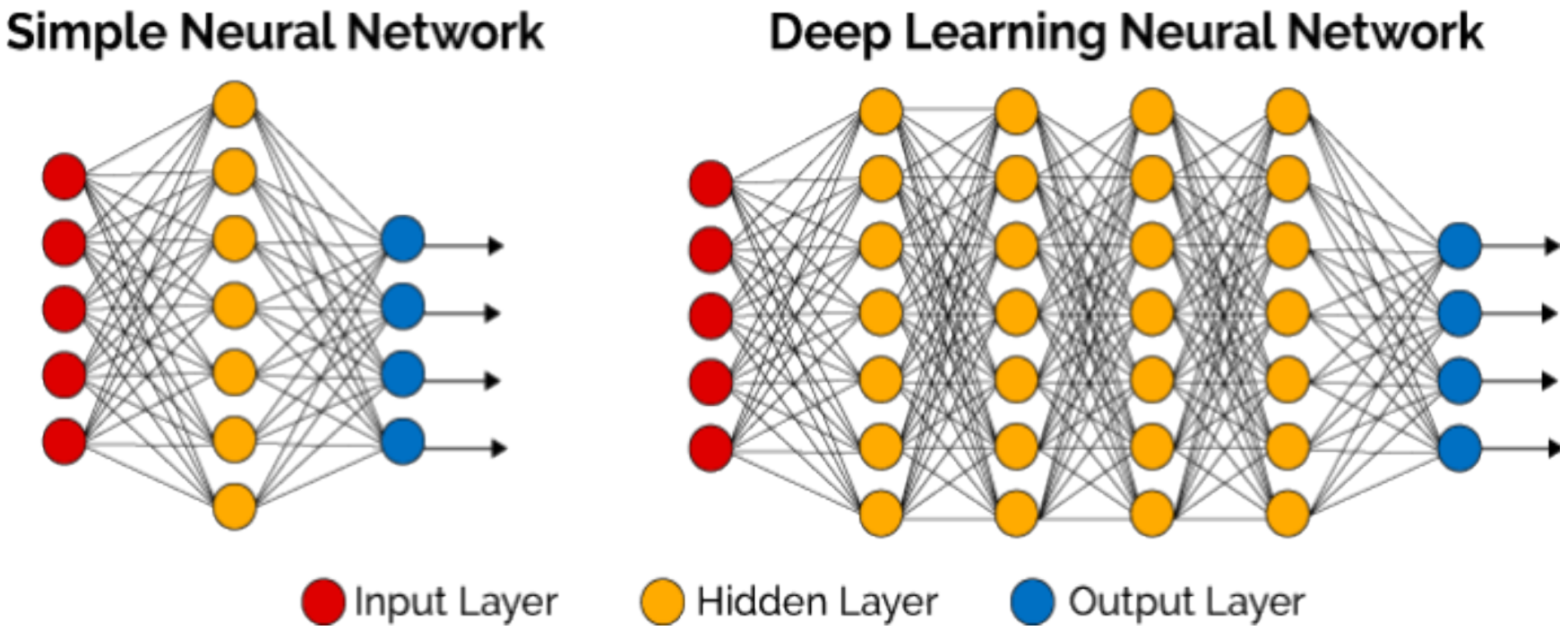
Deep Learning to rescue

DL solves these problems in representation learning by introducing representations that are expressed in terms of other, simpler representations.

DL enables the computer to build complex concepts out of simpler concepts.

Other challenges like understanding non-linearity, complex data-types, feature engineering can also be solved efficiently using Deep Learning.

MLP vs Deep Learning

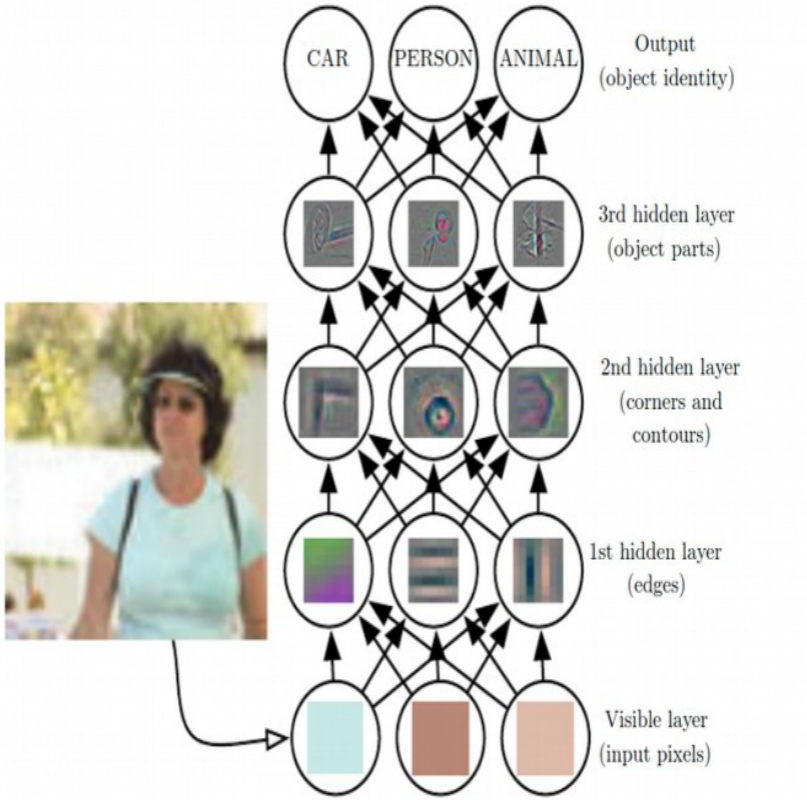


Illustrations of a Deep Learning Model

- Directly learning complex input–output mapping is very difficult
- Deep Learning (DL) solves this by breaking a complex problem into many simple steps
- Each step is handled by a layer in the network

Role of Hidden Layers

- Hidden layers automatically extract features
- They are called hidden because these features are not given explicitly
- Each layer learns a higher-level feature than the previous one



Feature Extraction Example (Image)

- **Input layer**: raw image pixels
- **First hidden layer**: detects **edges**
- **Second hidden layer**: detects **corners and contours**
- Higher layers: detect **object parts**

Depth of a model

- Depth means the number of layers in a model
- Deep learning focuses on learning the right data representation
- Each layer performs a small computation step
- Output of one layer becomes input to the next layer
- Deeper models can learn complex patterns
- Later layers build on results of earlier layers

Note: **The depth of a model refers to the number of layers that allow the network to learn complex representations through multiple sequential steps.**

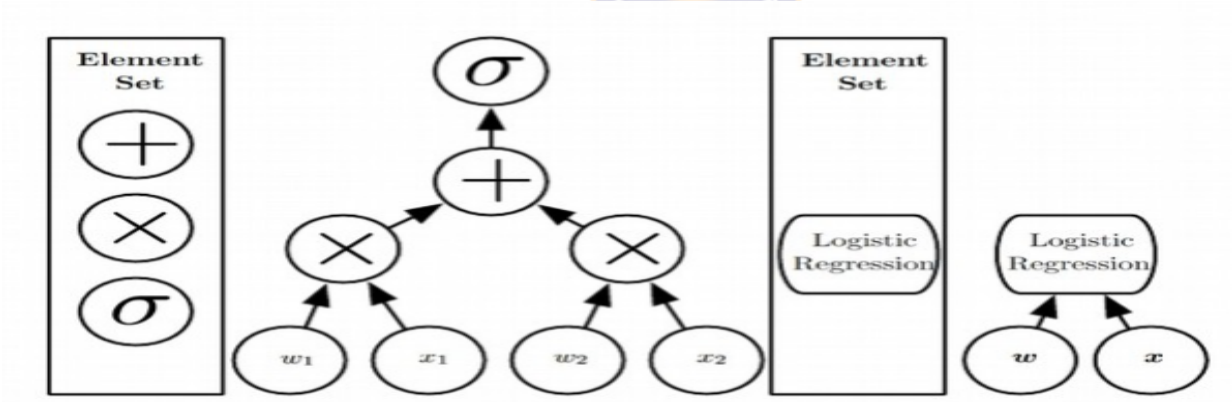
Depth of a model

There are two main ways of measuring the depth of a model

1. sequential steps
2. measuring depth (concept-based)

Approach 1

1. Model depth can be measured by the number of
   - sequential steps
2. It is the longest path from input to output
3. Each step represents a computation or operation
4. More steps → greater depth



Approach 2

- Depth can also be measured by how concepts depend on each other
- Simple concepts are learned first
- Complex concepts are learned by combining simpler ones
- Each level represents a higher-level abstraction
- Deeper models can refine understanding using earlier concepts

Note: In the second view, model depth is measured by the number of hierarchical concept levels used to represent and refine knowledge.

Depth of a model

2. Another approach, used by deep probabilistic models, regards the depth of a model as being not the depth of the computational graph but the depth of the graph describing how concepts are related to each other.

    1. This is because the system's understanding of the simpler concepts can be refined given information about the more complex concepts.

    2. For example, an AI system observing an image of a face with one eye in shadow may initially only see one eye. After detecting that a face is present, it can then infer that a second eye is probably present as well.

    3. The graph of concepts only includes two layers—a layer for eyes and a layer for faces—but the graph of computations includes 2n layers if we refine our estimate of each concept given the other n times.

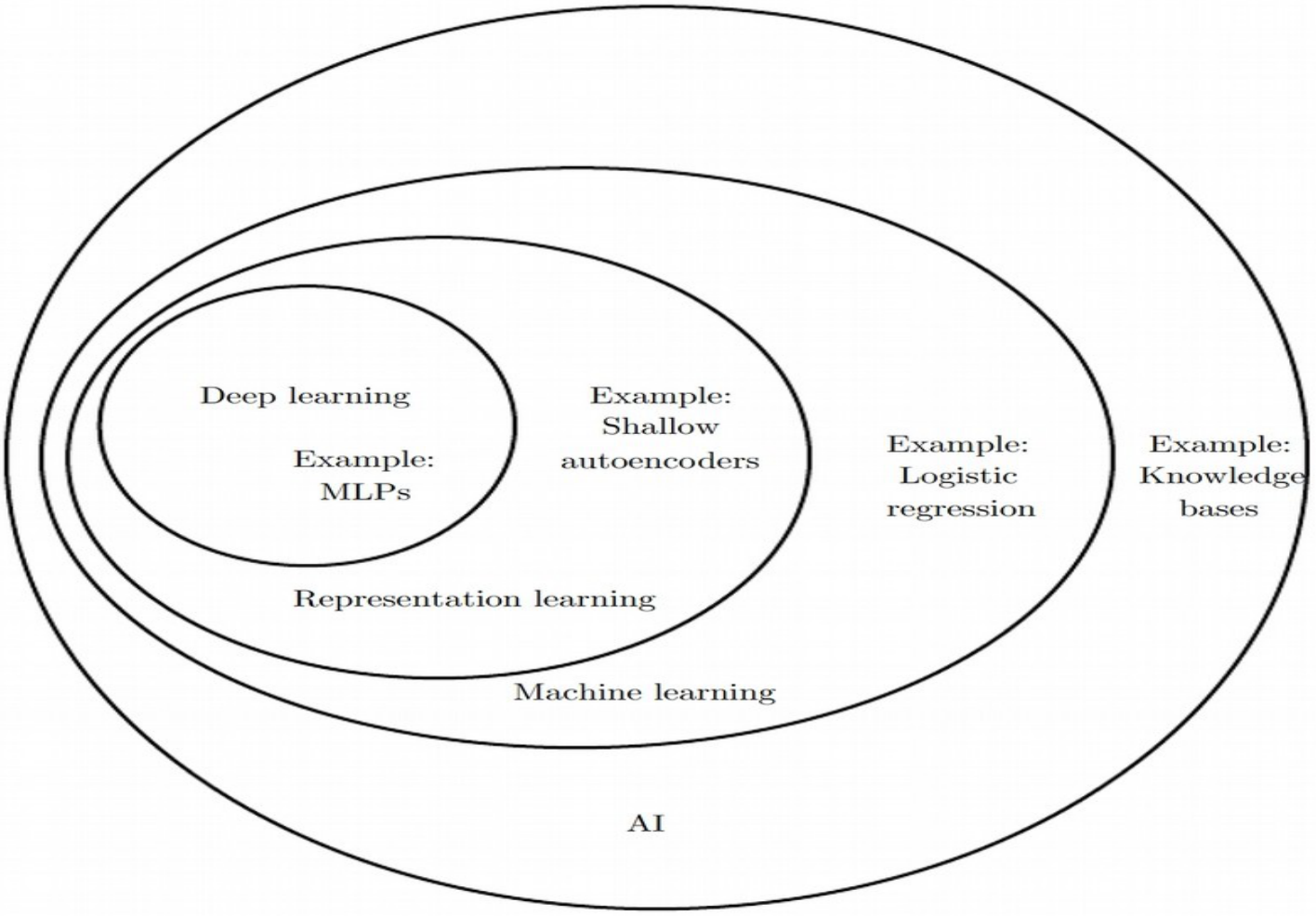
Depth of a model

Because it is not always clear which of these two views is most relevant, and There is no single correct value for the depth of an architecture, just as there is no single correct value for the length of a computer program.
Nor is there a consensus about how much depth a model requires to qualify as "deep."

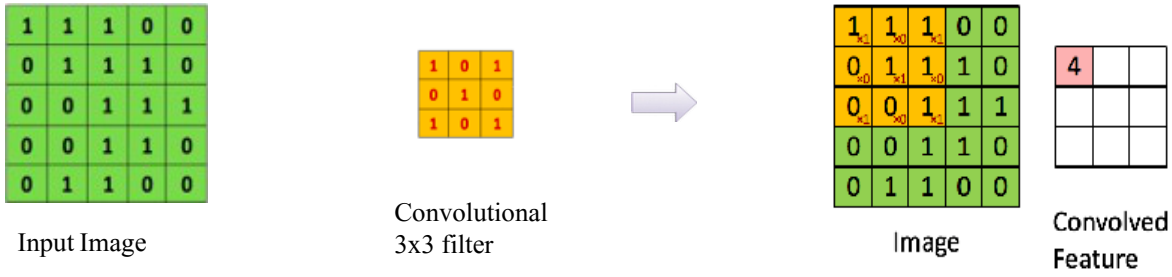
However, deep learning can safely be regarded as the study of models that either involve a greater amount of composition of learned functions or learned concepts than traditional machine learning does.

Deep learning is a particular kind of machine learning that achieves great power and flexibility by learning to represent the world as a nested hierarchy of concepts, with each concept defined in relation to simpler concepts, and more abstract representations computed in terms of less abstract ones.

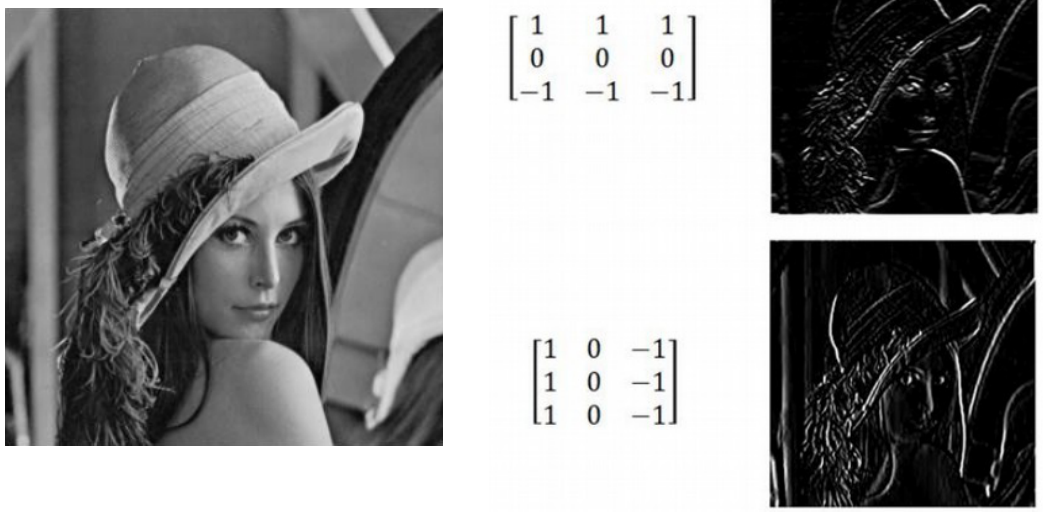
Deep Learning and AI



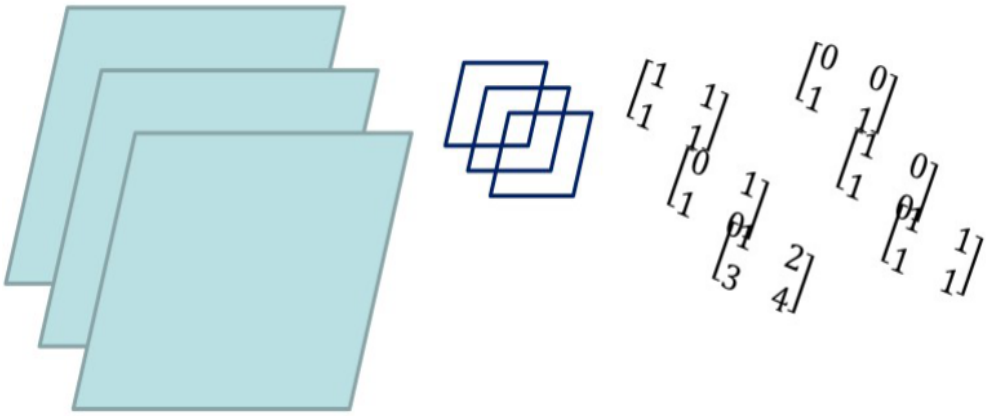
Convolutional Neural Networks (CNN)



CNN Filters



Convolution in 3 dimensions



$$
\begin{aligned}
= \quad & (1*0)+(1*0)+ \\
& (1*1)+(1*1)+ \\
& (0*1)+(1*0)+ \\
& (1*1)+(0*0)+ \\
& (1*1)+(2*1)+ \\
& (3*1)+(4*1) \\
= 13 &
\end{aligned}
$$

Even a 3D convolution gives a 2D output

Convolution in 3 dimensions

To summarize, the Conv Layer:

- Accepts a volume of size **W1×H1×D1**
- Requires four hyperparameters:
  - Number of filters **K**,
  - their spatial extent **F**,
  - the stride **S**,
  - the amount of zero padding **P**.
- Produces a volume of size **W2×H2×D2** where:
  - **W2=(W1 F+2P)/S + 1**
  - **H2=(H1 F+2P)/S + 1**
  - **D2=K**
- With parameter sharing, it introduces **F F D1** weights per filter, for a total of **(F F D1) K** weights and **K** biases.

A common setting of the hyperparameters is **F=3,S=1,P=1**. But varies for different types of problems and architectures.

CNN

Why do CNNs work?

The networks can be large and hence bias is minimized The weights are mostly zero because of convolution.
ReLU(Activation Function) and *Dropout* makes even fewer weights.
Hence, variance is minimized

When Context is important?

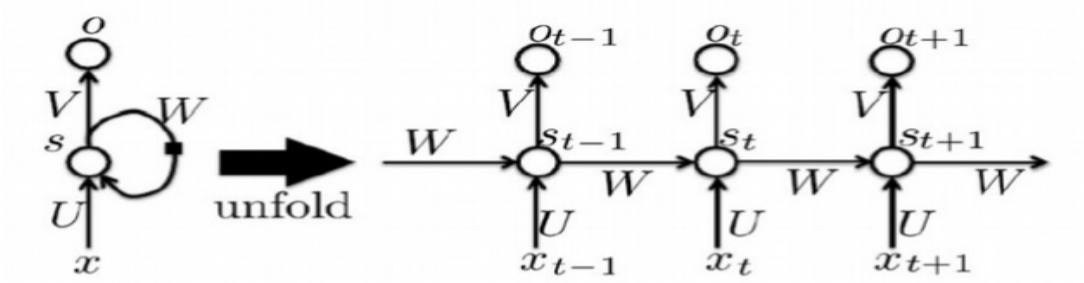
Sometimes, we need to have the contextual information to be able to perform Machine Learning predictions

E.g., Estimating the probabilities of words given the context The clouds are in the_____(sky – no further context is needed)

I fell in love with this French girl. My parents were against it initially as they were worried about the cultural differences. However, after they met her they realized how wonderful a person she is and agreed for our marriage. All that is left is convincing her parents. Here, I am booking my tickets to_____(A lot of context is needed)

Recurrent Neural Network (RNN)

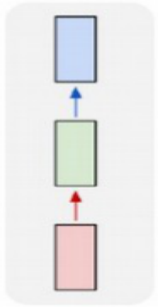
A recurrent neural network (RNN) is a type of artif icial neural network which uses sequential data or time series data. These deep learning algorithms are commonly used for ordinal or temporal problems, such as language translation, natural language processing (nlp), speech recognition, and image captioning; they are incorporated into popular applications such as Siri, voice search, and Google Translate.



$$\mathbf{a}_t = \mathbf{b} + W\mathbf{s}_{t-1} + U\mathbf{x}_t$$
$$\mathbf{s}_t = tanh(\mathbf{a}_t)$$
$$\mathbf{o}_t = \mathbf{c} + V\mathbf{s}_t$$
$$\mathbf{p}_t = softmax(\mathbf{o}_t)$$

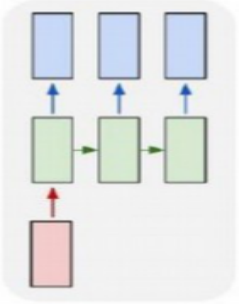
Types of recurrent neural networks

one to one



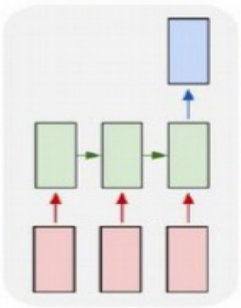
e.g. image classification

one to many



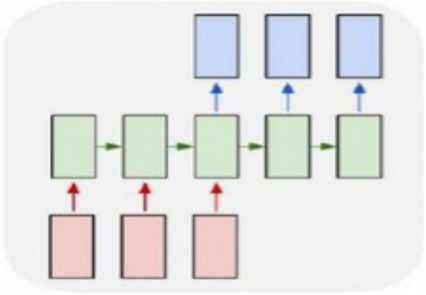
e.g. image captioning takes an image and outputs a sentence of words

many to one



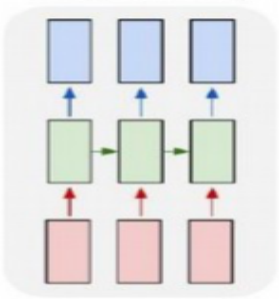
e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment

many to many



Machine Translation

many to many



Video classification where we wish to label each frame of the video

Build a model using reinforcement learning

Here you can discuss how to build a Model for Mario game using reinforcement learning.

- Setup A MARIO Environment
- Pre-process the Game for Applied AI
- Train the model to play MARIO TEST
- IT

Reference : https://python.plainenglish.io/build-an-ai-model-to-play-super-mario-7607b1ec1e17

Thank you

Any uestions?


STOPREC